PAVS Model Lifecycle & Delivery Strategy 2026

4-Phase Model Lifecycle

```
| ENABLEMENT |  ->  | PROFILING |  ->  | BENCHMARKING |  ->  | FINE-TUNING |

  Initial Setup      Performance          Validation             Optimization
  (1-2 weeks)        Analysis (3 days)    & Metrics (3 days)     (2-3 weeks)
```

Team Core Capabilities:

- **Model Enablement**: Off-the-shelf installation, dependency resolution, initial benchmarking
- **Performance Profiling**: Compute analysis, hotspot identification, bottleneck detection
- **Benchmarking & Validation**: Task metrics,Identify/suggest right model per domain/client,competitive analysis (GPU vs CPU vs NPU), CI/CD automation
- **Model Optimization**: Quantization (INT8, FP16), kernel optimization
- **Training Infrastructure**: Custom model training, domain-specific adaptation, fine-tuning on custom datasets
- **Deployment & Release**: Docker containerization, HuggingFace distribution, automated documentation

2026 Release Timeline

R1 2026 (Q1-Q3): Foundation

- Complete lifecycle framework for all models
- Streamline release logistics & CI/CD
- Deploy automated docs (GitHub Pages)

R2 2026 (Q4-Q6): Scale

- Single SDK installer (Nvidia/Intel style)
- 8 production-ready models (P0/P1)
- Automated benchmarking pipeline
- Quantization & Infra Beta (vLLM, ONNX-RT, Llama.cpp, Lemonade)
- Sample apps (ChatBot, Object Detection, GStreamer+ROCm)

R3 2026 (Q7-Q9): Expand

- 8 more release-ready models
- Enhanced quantization & infrastructure
- Model deployment & robotics simulator
- Scale fine-tuning operations

R4 2026 (Q10-Q12): Production

- 2 additional production models (20 total)
- Multi-segment deployment (Robotics, Healthcare, Industrial)
- Model training infrastructure
- 2027 strategy planning

Open Source Release & Distribution Strategy

GitHub-Based Development & Release

Repository Structure:

```
physical-ai-sdk/
├── models/
│   ├── mobilesam/
│   │   ├── gpu/          (Dockerfile, pyproject.toml, .venv)
│   │   ├── npu_gpu/      (hybrid environment)
│   │   └── npu/          (NPU-optimized)
│   ├── yolov12/
│   ├── whisper/
│   └── [20 models...]
├── benchmarks/           (automated CI/CD tests)
├── docs/                 (auto-generated)
└── tools/
    ├── profilers/        (Lemonade, ROCm profilers)
    └── optimizers/       (quantization, training)
```

CI/CD Automation Pipeline:

```
Developer Branch → Pull Request → Auto Tests
     ↓                ↓              ↓
  Regression       Smoke Tests    Benchmarking
     ↓                ↓              ↓
Release Branch → QA Validation → GitHub Release
     ↓                               ↓
Documentation Update           HuggingFace/tipp Push
```

Release Workflow:

```
Model Complete → Benchmarking → Optimization
     ↓               ↓              ↓
  Package       Verify Metrics   Test Deploy
     ↓               ↓              ↓
GitHub Release → HuggingFace Push → Docs Update
     ↓                               ↓
Community Access              Analytics & Feedback
```

HuggingFace Hub Distribution

What We Publish:

- 🤗 **Pre-optimized Model Weights**: ONNX, TensorRT, ROCm-ready
- 📦 **Quantized Variants**: INT8, FP16 for edge deployment
- 🐳 **Docker Images**: GPU/NPU/Hybrid ready-to-run containers
- 📊 **Benchmark Results**: Verified performance metrics included
- 📝 **Model Cards**: Documentation, usage examples, hardware requirements

HuggingFace Advantages:

- ✅ **Public Discovery**: Searchable by AI/robotics community
- ✅ **Version Control**: Track model iterations & improvements
- ✅ **Direct Download**: `pip install` or one-click deployment
- ✅ **Community Engagement**: Issues, feedback, contributions
- ✅ **Ecosystem Integration**: Works with Transformers, ONNX-RT, vLLM

Infrastructure Tools (Open Source Ready):

- **vLLM**: Efficient LLM serving & inference
- **Llama.cpp**: Optimized CPU/GPU inference
- **ONNX Runtime**: Cross-platform model deployment
- **Lemonade Profiler**: Performance analysis tool